Comparing Forecasters for Serverless Invocation Provisioning

Mojtaba Jafari
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
mojtaba.jafari@aut.ac.ir

Ehsan Moradi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
your.email@aut.ac.ir

Amir Kalbasi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
kalbasi@aut.ac.ir

***Abstract*—A cold start is the reason to forecast serverless invocations: a request that finds no warm container waits while one is created. The Azure Functions trace used here records per-minute counts, not container starts, so we count the containers a forecast leaves short and the containers it leaves idle, assuming one container serves 10 invocations a minute. Six forecasters and several references are compared on 80 functions in four workload classes, with an hourly 60-minute forecast. A first scoring made Chronos and ETS look wasteful because a forecast of 0.01 invocations still prepares a container. After forecasts below one half are set to zero, their median idle capacity falls from thousands of container-minutes to about a hundred or fewer. A previous-hour copy, seasonal naive, and Holt–Winters have the lowest error and are close to each other, and neither Chronos nor an LSTM beats them. A high quantile of recent load cuts the median shortfall from 75 to 18.5 container-minutes at about 730 extra container-minutes, and a high Chronos quantile does the same. A class-specific choice made on two days was not distinguishable from seasonal naive on the next weekend. A reactive ten-minute keep-alive that reads actual counts had a far lower shortfall than any hourly forecast.**

***Index Terms*—serverless computing, workload forecasting, cold start, provisioning**

I. Introduction

Serverless computing abstracts infrastructure management through fine-grained, event-driven execution, yet its elasticity remains constrained by cold-start latency. When an incoming invocation finds no warm instance, it incurs a substantial initialization delay. While keeping idle containers warm eliminates this penalty, it consumes costly memory resources. Serverless capacity provisioning thus contends with an inherent dilemma: minimizing user-facing cold starts while curtailing wasteful idle memory footprint.

Predictive provisioning addresses this tension by forecasting invocation demand to pre-warm containers proactively. However, a fundamental disconnect exists between standard time-series evaluation and practical cloud capacity management. Conventional statistical metrics, such as sMAPE or MSE, treat errors symmetrically. In production environments, forecast errors carry strictly asymmetric consequences: under-prediction yields container shortfalls and cold starts, whereas over-prediction leaves instances idle. Crucially, container allocation operates as a discrete step function, meaning that even infinitesimal continuous forecasts can trigger full container allocations. Consequently, models optimized purely for statistical loss can produce severely distorted provisioning decisions.

Compounding this challenge is the extreme heterogeneity of real-world serverless workloads. Production traces, such as Microsoft Azure Functions [1], reveal heavily skewed behaviors across sparse dormancy, diurnal patterns, and unpredictable bursts. Prior studies typically embed forecasting models inside complex autoscaling controllers, entangling pure predictive accuracy with ad-hoc scheduling heuristics. It remains an open question how diverse forecasters perform when cleanly isolated under an explicit capacity provisioning model.

In this work, we conduct a systematic, capacity-driven empirical comparison of invocation forecasters across four representative workload archetypes (sparse, daily, bursty, and other) sampled from the Azure trace. Across 80 functions, we evaluate six core strategies spanning simple training-free heuristics (last-value, trailing percentiles, seasonal naive), classical smoothing (seasonal Holt–Winters / ETS), deep recurrent networks (LSTM), and zero-shot foundation models (Chronos-Bolt [2]), alongside reactive baselines. We evaluate point accuracy alongside container shortfall (an operational proxy for cold starts) and memory-weighted idle costs over 60-minute horizons.

Our empirical investigation reveals three central findings. First, continuous forecasters require an explicit cutoff for fractional predictions (< 0.5); without it, discrete container rounding artificially inflates idle capacity into thousands of container-minutes. Second, lightweight heuristics and Holt–Winters match or surpass deep learning and foundation models in point accuracy, while cold-start shortfall is governed by quantile headroom rather than model complexity. Third, per-archetype model selection fails to outperform simple baselines under weekend distribution shifts, and hourly proactive horizons inherently lag behind fine-grained reactive keep-alive mechanisms.

II. Related Work

A. Workload Characterization and Keep-Alive Strategies

Cold-start mitigations broadly encompass container caching, runtime optimizations, and predictive resource provisioning

xxxx-..-....-....-x/xx/$xx.00 ©2026IEEE

[3]. In their seminal characterization of Microsoft Azure Functions, Shahrad et al. revealed extreme workload skewness: most functions are invoked infrequently, inter-arrival times span multiple orders of magnitude, and memory allocation is heavily concentrated [1]. Multi-tenant traces analyzed by Joosen et al. corroborated this skewness, noting that invocation forecasting remains an open challenge [4]. To balance latency and cost, Shahrad et al. introduced a hybrid histogram policy that tunes idle timeouts, while Lambda-Warmer adapts keep-alive windows from observed container reuse [5]. Nonetheless, keep-alive mechanisms remain inherently reactive, leaving platforms vulnerable to cold starts when bursts follow idle periods and motivating proactive workload forecasting.

B. Predictive Autoscaling and Proactive Schedulers

To anticipate arrival surges before cold starts occur, recent research embeds predictive modeling directly into serverless autoscaling loops. Rather than evaluating forecasters in isolation, contemporary frameworks typically bind predictions to custom scaling controllers. For instance, Zhang et al. introduced AAPA [6], an archetype-aware Kubernetes autoscaler that classifies invocation histories into spike, periodic, ramp, and stationary patterns to assign tailored policies with uncertainty bounds. Similarly, Daraghmeh et al. framed container provisioning as a classification task, deploying machine learning classifiers to govern scale-up and scale-down decisions [7].

Beyond classification, hybrid and control-theoretic frameworks have gained traction. Shiekhani et al. combined ARIMA with random forests for container pre-warming on OpenWhisk [8], while Nguyen et al. paired Fourier forecasting with model predictive control [9]. Deep models have also predicted cold-start events for provisioned concurrency [10], and Transformers have targeted pre-warming on Azure traces [11]. However, such end-to-end evaluations entangle forecasting accuracy with scheduling heuristics and container reuse. In contrast, we isolate the forecasters under a unified capacity protocol to evaluate their direct impact on container provisioning gaps.

C. Time-Series Forecasters: From Heuristics to Foundation Models

Classical statistical baselines, including seasonal naive repetition and Holt–Winters smoothing, remain ubiquitous benchmarks. Recently, time-series foundation models have emerged as an attractive zero-shot alternative. Ansari et al. introduced Chronos [2], which tokenizes continuous series to train transformer models for zero-shot forecasting. Chronos-Bolt emits multi-horizon quantile predictions in one pass, while Chronos-2 adds multivariate support [12]. Concurrent models such as TimesFM [13] and Moirai [14] exhibit similar zero-shot versatility. While foundation models eliminate per-function training, their capacity-aware efficiency remains uncharacterized. We evaluate Chronos-Bolt against classical baselines on error, and quantify what idle capacity its quantiles expend to suppress shortfalls.

III. Setup

Our empirical study evaluates the public Microsoft Azure Functions dataset [1], which records per-minute invocation counts over 14 consecutive days (treating missing days as 1,440 zeros). To capture the spectrum of production behaviors, we classify functions into four distinct archetypes based on profiling statistics from days 1–10: *sparse* functions exhibit zero invocations in at least 90% of recorded minutes (representing 54,502 of the 68,394 fleet functions); *daily* functions exhibit strong diurnal periodicity, identified by a lag-1440 autocorrelation exceeding 0.30; *bursty* functions capture volatile, non-periodic arrivals where the coefficient of variation reaches at least 2.05 (the 75th percentile among non-sparse functions); and *other* functions comprise the remaining non-periodic workloads. We sample 20 eligible functions from each archetype (seed 42), requiring at least one invocation during days 1–10 and days 11–14, yielding an 80-function benchmark.

Forecasts are generated for a 60-minute lookahead horizon and re-issued hourly across days 11–14, producing 96 non-overlapping evaluation windows per function. Days 1–10 serve as historical training and profiling data, while days 11–14 constitute the held-out test period. In addition, weekend days 13–14 are analyzed separately to assess whether model rankings established on weekdays (days 11–12) persist under temporal distribution shifts. For context input, seasonal Holt–Winters, Chronos, and the LSTM observe a 24-hour historical window (1,440 minutes), whereas recency heuristics (last-value and trailing percentiles) condition strictly on immediate observations.

The primary evaluation compares six diverse forecasting strategies: 1) *Seasonal Naive* repeats the corresponding 60 minutes from the previous day when lag-1440 autocorrelation exceeds 0.30, defaulting to repeating the preceding hour otherwise; 2) *Last Value* projects the single latest observed minute uniformly across the hour; 3) *Trailing P95* projects the 95th percentile of the preceding 10 minutes; 4) *ETS (Holt–Winters)* employs additive exponential smoothing with daily seasonality, optimizing level, trend, and seasonal smoothing parameters per function via one-step MSE grid search on days 1–10 and updating internal states recursively; 5) *Chronos* utilizes the median point forecast from pretrained `amazon/chronos-bolt-small` [2], deployed zero-shot without trace-specific fine-tuning; and 6) *LSTM* employs a shared single-layer architecture (32 units, linear output) trained on sliding windows from days 1–8 with MAE loss, early-stopped on days 9–10, with inputs scaled by context mean.

Several reference baselines provide additional comparative context: unconditional *Previous Hour* and *Always-Previous-Day* copies; intermittent-demand models (Croston and TSB [15], smoothing $\alpha = 0.1$); high-quantile variants including Chronos (80th and 90th percentiles) and an LSTM trained under pinball loss ($\tau = 0.9$); and multiplicative safety margins ($1.25\times$, $1.5\times$, $2.0\times$). In addition, a *reactive keep-alive* serves as an operational benchmark: updated per-minute, it

maintains warm container capacity matching the peak usage of the preceding 10 minutes, representing an idealized reactive controller rather than a fixed 60-minute lookahead forecast.

To translate continuous invocation forecasts into discrete resource allocations, our capacity model assumes a nominal concurrency of 10 invocations per container-minute. Crucially, raw continuous predictions below 0.5 invocations are clamped to zero; without this threshold, infinitesimal non-zero predictions (e.g., 0.01) round up to a full container, artificially distorting idle capacity. For an actual invocation count y_t and forecast $\hat{y}_t$ (clipped at zero), required containers $r_t = \lceil y_t/10 \rceil$ and prepared containers $p_t = \lceil \hat{y}_t/10 \rceil$ determine the operational metrics:

$$S = \sum_t \max(0,\, \lceil y_t/10 \rceil - \lceil \hat{y}_t/10 \rceil)\,, \tag{1}$$

$$O = \sum_t \max(0,\, \lceil \hat{y}_t/10 \rceil - \lceil y_t/10 \rceil)\,. \tag{2}$$

Here, S measures *container shortfall* (in container-minutes), serving as an operational proxy for cold starts since the trace lacks explicit container start events. Conversely, O measures *idle capacity* (extra container-minutes). To quantify the economic footprint of over-provisioning, we calculate memory-weighted idle overhead ($O \times$ memory) using each function's application-level median allocated memory from days 1–10 (sample median 144 MB, range 90–518 MB). Because container provisioning follows discrete steps, numerical deviations that stay within the same container boundary incur neither penalty nor waste. A third metric, *new containers*, defined as $\sum_t \max(0,\, r_t - \max(p_t, r_{t-1}))$, accounts for container reuse: an unabsorbed surge spanning multiple consecutive minutes counts once per instantiated container rather than accumulating per minute.

For statistical comparison, forecast accuracy is measured using symmetric Mean Absolute Percentage Error (sMAPE) per minute:

$$\mathrm{sMAPE}_t = 100 \cdot \frac{2|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|}, \tag{3}$$

where minutes with zero actual and predicted demand contribute zero. Because a few high-volume functions dominate fleet aggregates, we summarize per-function distributions using class and fleet medians. Statistical significance is evaluated via 95% percentile bootstrap confidence intervals (and paired differences) computed from 2,000 within-class resamples.

IV. Evaluation

A. Scoring cutoff

A first scoring of the same forecasts gave ETS, the LSTM, and Chronos median extra capacities of 2,101.5, 3,640.5, and 3,281.0 container-minutes. Those values came from the ceiling. On the sparse functions the Chronos forecast was positive on 88% of minutes and below 0.5 on almost all of them, and ETS was positive on 50%. Each such minute prepared a whole container. After the cutoff the same medians are 112.5, 0.5, and 1.5. Every number below uses the cutoff for every model. Fixed forecasts such as seasonal naive and the trailing percentile are barely affected.

B. Error

Table I summarizes days 11–14. Previous hour has the lowest overall median sMAPE, 4.7%. Seasonal naive has 7.1% and ETS 8.2%, and the paired median difference between ETS and seasonal naive is 0.0 points per function (95% interval -0.0 to 0.7). Chronos has 13.5%, the LSTM 17.0%, last-value 19.2%, and the trailing P95 24.7%. The intervals are wide. Seasonal naive has 7.1% with an interval of 2.7 to 15.3, so differences of a few points among the first three are within noise. The class split shows who is good where. ETS is best on daily functions, 2.1% against 3.9% for seasonal naive. Chronos is best on other functions, 7.8% against 12.9%, and Chronos and the LSTM are slightly ahead of seasonal naive on bursty functions, 34.8% and 34.6% against 41.0%. Chronos and the LSTM tie on sparse functions at 0.7%, because both predict almost nothing. The autocorrelation gate in seasonal naive does not matter overall: always repeating the previous day gives 9.6% and always repeating the previous hour 4.7%, so on these minute counts yesterday's shape is not what makes the copy accurate. Fig. 1 shows one daily and one sparse function.

C. Shortfall and idle capacity

Every forecaster is short on most functions (Table II): from 66% of functions for the trailing P95 to 90% for previous hour. Seasonal naive has a median shortfall of 75 container-minutes (interval 29 to 169) and 80.5 extra. The trailing P95 has a median shortfall of 18.5 (4 to 52) and 729 extra (369 to 2,177). The paired difference to seasonal naive is -35.5 short container-minutes (-108.5 to -15.0) and $+426.5$ extra (214 to 1,228). New containers fall from 25.5 to 12.5. In application memory, the extra capacity grows from 15.3 to 135.4 thousand megabyte-minutes. Chronos at its 90th percentile gives almost the same point: 18.5 short, 754.5 extra, 8.5 new containers. ETS (52.5, 112.5) has a somewhat lower shortfall than seasonal naive and somewhat more idle capacity. Chronos median (253.5, 1.5) and the LSTM (303.5, 0.5) trade the other way: they leave little idle capacity and are short on more minutes.

Fig. 2 gives headroom to each forecaster and pools functions, so heavy functions count in proportion to their load. The curves are close. Moving up a quantile of recent load or of Chronos buys shortfall at a slightly lower price than multiplying seasonal naive by 1.25 or 1.5. No amount of headroom brings the pooled shortfall below about 14% of the required container-minutes, because the bursty functions dominate it. An LSTM trained to a 0.9 level, one run, lands near the same curve with more idle capacity: 20.5 short and 1,205.5 extra. The reactive 10-minute keep-alive is far below every curve: a median of 6 short container-minutes and 1,038 extra, with a pooled shortfall of 1.6% at 43% extra. It reads the actual counts each minute, so it is not a competing forecaster. It shows what an hourly forecast interval gives up.

D. Class-specific choices and the test days

Table III compares single forecasters with two class-specific picks. A pick takes, for each class, the forecaster that is best on days 11–12. One pick ranks by shortfall and then by idle capacity. The other ranks by short plus extra container-minutes. Days 13 and 14 are a weekend: the sampled functions have 1.23 and 0.99 million invocations, against 3.41 and 3.06 million on days 11 and 12, and the same drop appears on days 6 and 7. The medians for days 13 and 14 are therefore lower for every forecaster and are not like-for-like with days 11 and 12. The shortfall-first pick chooses ETS for sparse and the trailing P95 for the other three classes, and on days 13 and 14 it matches always using the trailing P95. The short-plus-extra pick chooses the LSTM for sparse, ETS for daily, Chronos for bursty, and the trailing P95 for other functions. It has a median of 5 short and 0 extra on days 13 and 14, against 12 and 15 for seasonal naive, but the paired median differences are 0 short (interval 0 to 0) and 0 extra (-2.0 to 0.0), so the gain is not distinguishable from zero. Twenty functions per class and two selection days are too little to support a different forecaster for each class.

E. Capacity and weighting

A fixed capacity of 10 invocations per container-minute is arbitrary. The median average duration of the sampled functions is 326 ms, so one sequential container could serve about 184 invocations a minute. Capacity from each function's own average duration, with a floor of one, gives for seasonal naive 32.5 short and 38.0 extra, for the trailing P95 5.0 and 246.0, for ETS 29.0 and 34.5, for Chronos 69.0 and 0.0, and for the LSTM 88.5 and 0.0. The ordering is the same as at 10. Weighting each function by the number of eligible functions in its class, so that the sparse class counts for most of the fleet, gives medians of 17 short and 17 extra for seasonal naive, 15 and 129 for the trailing P95, 16 and 12 for ETS, and 24 and 0 for Chronos. The All column in the tables is a statistic of the sample, not of the fleet.

F. Limits

The trace is from 2019 and contributes 80 sampled functions, 20 per class, not a live deployment. The class thresholds were not varied. Eligibility required at least one invocation on days 11–14, so a function that goes silent in the test window never enters the sample. The counts are per minute, so S is a capacity gap, and new containers is a proxy for cold starts, not a measurement on a platform. Containers are counted per function with one invocation at a time, while Azure scales per application. Memory is the allocated size of the application. The forecasts are issued once an hour and held, which favours methods that copy the past hour or day and penalizes reactive ones. Only the fixed 10-minute keep-alive is evaluated, not the histogram policy of Shahrad et al. [1]. The cutoff of 0.5 was set once, before scoring, and applied to all models. The LSTM is one run of 12 epochs whose validation loss was still falling, and no per-class learned model was trained.

TABLE I
MEDIAN SMAPE, SHORT CONTAINER-MINUTES, AND EXTRA CONTAINER-MINUTES ON DAYS 11–14.

Model	Metric	Sparse	Daily	Bursty	Other	All
Seasonal naive	sMAPE (%)	0.9	3.9	41.0	12.9	7.1
	Short	13.0	45.0	718.0	150.0	75.0
	Extra	13.0	43.0	719.0	150.5	80.5
Previous hour	sMAPE (%)	1.1	3.3	41.0	12.9	4.7
	Short	15.5	37.0	718.0	150.0	66.0
	Extra	15.5	43.0	719.0	150.5	69.5
Last value	sMAPE (%)	0.9	37.8	48.2	11.9	19.2
	Short	17.0	1,150	763.5	99.5	410.0
	Extra	0.0	31.0	593.5	93.5	63.5
Trailing P95	sMAPE (%)	3.8	134.4	74.6	12.9	24.7
	Short	15.0	0.0	143.0	0.0	18.5
	Extra	87.5	4,386	2,183	98.0	729.0
ETS	sMAPE (%)	0.9	2.1	55.1	13.7	8.2
	Short	13.0	45.0	615.5	47.0	52.5
	Extra	9.5	8.0	969.5	166.5	112.5
LSTM	sMAPE (%)	0.7	39.9	34.6	12.0	17.0
	Short	19.0	1,150	924.0	0.0	303.5
	Extra	0.0	0.0	125.5	142.5	0.5
Chronos	sMAPE (%)	0.7	34.3	34.8	7.8	13.5
	Short	19.0	1,088	996.5	0.0	253.5
	Extra	0.0	0.0	48.0	144.0	1.5

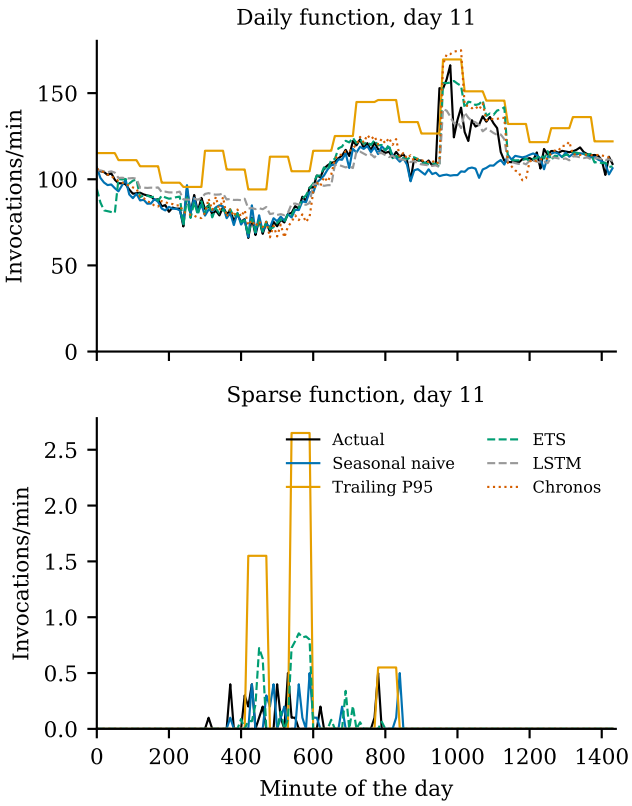


Fig. 1. Day 11 for one daily and one sparse function, in 10-minute means. The other forecasters follow the same shape or stay near zero.

V. CONCLUSION

The goal is fewer cold starts. This trace cannot count them, so the evidence is the container shortfall and the idle capacity under a stated capacity, with idle capacity also weighted by application memory. Three findings stand. First, the idle cost

TABLE II

ALL 80 FUNCTIONS, DAYS 11–14. MEDIANS. NEW IS NEW CONTAINERS, MB IS THOUSANDS OF EXTRA MEGABYTE-MINUTES, AND SHORT% IS THE SHARE OF FUNCTIONS WITH ANY SHORTFALL. THE STAR MARKS A REACTIVE REFERENCE THAT READS ACTUAL COUNTS.

Model	sMAPE	Short	Extra	New	MB	Short%
Seasonal naive	7.1	75.0	80.5	25.5	15.3	88
Previous hour	4.7	66.0	69.5	25.0	12.6	90
Last value	19.2	410.0	63.5	141.5	12.4	88
Trailing P95	24.7	18.5	729.0	12.5	135.4	66
ETS	8.2	52.5	112.5	21.0	17.0	79
LSTM	17.0	303.5	0.5	141.5	0.1	84
LSTM, tau 0.9	35.3	20.5	1,206	6.0	194.6	68
Chronos	13.5	253.5	1.5	127.0	0.4	85
Chronos q90	22.9	18.5	754.5	8.5	111.2	68
TSB	17.3	224.0	10.0	111.0	1.4	84
Croston	32.5	233.0	10.0	111.0	1.4	81
Keep-alive 10 min*	–	6.0	1,038	6.0	157.2	69

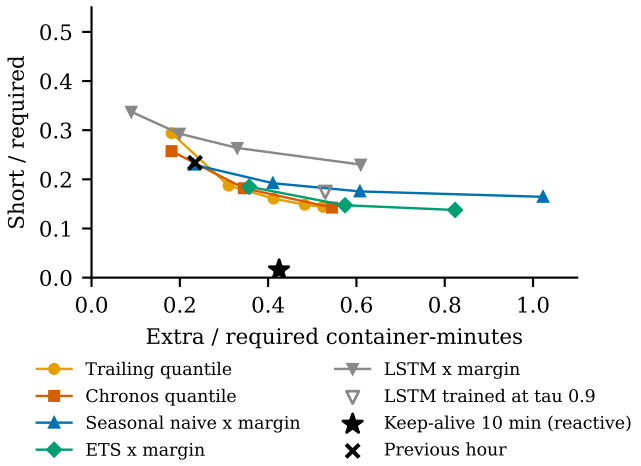


Fig. 2. Pooled short and extra container-minutes as a share of the required container-minutes on days 11–14, with headroom added to each forecaster. Closer to the lower left is better.

of continuous forecasts depended on a cutoff for tiny forecasts; with it, ETS, Chronos, and an LSTM are not wasteful. Second, a previous-hour copy, seasonal naive, and Holt–Winters are the most accurate and statistically close, and neither Chronos nor the LSTM beats them overall, although Chronos is best on other functions and ETS on daily ones. Third, shortfall is bought with headroom, and a high quantile of recent load buys it at about the same price as a high quantile of Chronos: a median shortfall of 18.5 container-minutes instead of 75, for about 730 extra. A class-specific choice made on two days was not distinguishable from seasonal naive, and a reactive 10-minute keep-alive had a far lower shortfall than any hourly forecast. The next measurement is shorter forecast intervals with rolling re-forecasting, and cold starts recorded on a real platform.

TABLE III

MEDIANS FOR ALL FUNCTIONS ON DAYS 11–12 AND ON THE WEEKEND DAYS 13–14. THE TWO PICKS CHOOSE A FORECASTER PER CLASS ON DAYS 11–12.

	Days 11–12			Days 13–14		
Policy	sMAPE	Short	Extra	sMAPE	Short	Extra
Seasonal naive	8.4	48.5	59.0	2.7	12.0	15.0
Trailing P95	32.6	13.0	694.5	16.2	0.5	120.5
ETS	9.4	30.5	55.5	7.8	8.5	39.5
LSTM	21.2	271.5	0.5	12.3	70.5	0.0
Chronos	15.7	199.5	1.0	7.6	47.0	0.0
Pick: shortfall first	26.4	12.5	694.5	12.7	0.0	120.0
Pick: short plus extra	7.8	37.5	2.0	1.6	5.0	0.0

REFERENCES

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 205–218.

[2] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado *et al.*, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=gerNCVqqtR

[3] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, "Cold start latency in serverless computing: A systematic review, taxonomy, and future directions," *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–36, 2025.

[4] A. Joosen, A. Hassan, M. Asenov, R. Singh, L. Darlow, J. Wang, and A. Barker, "How does it function? characterizing long-term trends in production serverless workloads," in *Proceedings of the 2023 ACM Symposium on Cloud Computing*, 2023, pp. 443–458.

[5] Y. Singh and R. Shrestha, "Lambda-warmer: A measurement-driven adaptive framework for cost-aware cold-start mitigation in AWS lambda," *Future Generation Computer Systems*, vol. 185, p. 108681, 2026.

[6] G. Zhang, S. Vippagunta, R. Nandagopal, S. Raman, J. Xu, M. Pfeiffer *et al.*, "AAPA: An archetype-aware predictive autoscaler with uncertainty quantification for serverless workloads on Kubernetes," *arXiv preprint arXiv:2507.05653*, 2025.

[7] M. Daraghmeh, Y. Jararweh, and A. Agarwal, "Leveraging machine learning and feature engineering for optimal data-driven scaling decision in serverless computing," *Simulation Modelling Practice and Theory*, vol. 140, p. 103090, 2025.

[8] L. Shiekhani, H. Wang, W. Shi, J. Liu, Y. Qiu, C. Gu, and W. Ding, "The hybrid model: Prediction-based scheduling and efficient resource management in a serverless environment," *Applied Sciences*, vol. 15, no. 14, p. 7632, 2025.

[9] C. Nguyen, M. Bhuyan, and E. Elmroth, "Taming cold starts: Proactive serverless scheduling with model predictive control," *arXiv preprint arXiv:2508.07640*, 2025.

[10] N. S. Kumar and S. S. Samy, "Cold start prediction and provisioning optimization in serverless computing using deep learning," *Concurrency and Computation: Practice and Experience*, vol. 37, no. 4–5, 2025.

[11] A. S. F. M. Mouen, J. L. Zeutouo, and V. K. Tchendji, "Transformer-based model for cold start mitigation in FaaS architecture," *arXiv preprint arXiv:2504.11338*, 2025.

[12] A. F. Ansari, O. Shchur, J. Küken, A. Auer, B. Han, P. Mercado *et al.*, "Chronos-2: From univariate to universal forecasting," *arXiv preprint arXiv:2510.15821*, 2025.

[13] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 10 148–10 167.

[14] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified training of universal time series forecasting transformers," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 53 140–53 164.

[15] R. H. Teunter, A. A. Syntetos, and M. Z. Babai, "Intermittent demand: Linking forecasting to inventory obsolescence," *European Journal of Operational Research*, vol. 214, no. 3, pp. 606–615, 2011.